

Machine Learning for Mortgage Analysis

Loan Approval Classification and Approved Loan Amount Regression

Applied Data Science Final Project

Authors: Reza Kazemeiny Moghaddam and Saleh Mohammad Hasani

Date: July 2026

Project scope. This report combines two connected machine learning tasks based on the 2017 Home Mortgage Disclosure Act data. The first task predicts mortgage application approval. The second task predicts the loan amount for applications that were already approved. The report covers data preparation, model design, results, error analysis, explainability, fairness limits, discussion, and the final conclusion.

Abstract

Mortgage lending is an important financial process. It affects applicants, lenders, regulators, and local communities. Mortgage data contain many kinds of information, such as applicant income, loan purpose, property type, location, and lender identity. These variables have complex relations, so mortgage analysis is a useful problem for machine learning.

This project studies two tasks with the 2017 Home Mortgage Disclosure Act (HMDA) data. The first task is binary classification. It predicts whether a mortgage application is approved. The second task is regression. It predicts the loan amount for an application that was already approved. The raw file had 14,285,496 rows and 78 columns. A chunk-based cleaning process removed columns with high missingness, administrative fields, coded duplicates, post-decision information, and invalid rows. The final ready table had 8,064,813 rows and 29 columns. Two 500,000-row samples were then created for classification and regression.

For classification, Logistic Regression, Random Forest, CatBoost, and LightGBM were tested under three class-balancing designs. Six deep learning architectures were also studied. The best classification model was CatBoost trained on the full class-weighted data. It reached a ROC-AUC of 0.8253, balanced accuracy of 0.7439, and F1-score of 0.7573. The majority-vote ensemble did not improve the best individual result.

For regression, the project compared linear models, tree models, gradient boosting, and deep tabular models. The official final model was a weighted blend of CatBoost, LightGBM, and XGBoost with weights 0.60, 0.20, and 0.20. On the locked test set, this blend reached a Mean Absolute Error (MAE) of 61.511631 thousand US dollars, or about 61,512 dollars. It also reached an RMSE of 125.788460 and an R^2 of 0.712206. Error analysis showed that large loans were much harder to predict and were often underpredicted.

The results show that gradient-boosted trees were the strongest choice for these large tabular datasets. Deep learning models were useful for comparison, but they did not replace the best boosting models. The project also shows why data leakage control, fixed evaluation splits, clear metric scopes, human review, and careful fairness language are important in financial machine learning.

Keywords: HMDA, mortgage lending, classification, regression, CatBoost, LightGBM, XGBoost, tabular deep learning, fairness, explainability.

Contents

Abstract	i
1 Introduction	1
1.1 Background	1
1.2 Project objectives	1
1.3 Why the problem is important	1
2 Related Work	1
2.1 Classical models for tabular data	1
2.2 Gradient boosting	2
2.3 Deep learning for tabular data	2
3 Data Source and Common Preparation	2
3.1 HMDA 2017 data	2
3.2 Cleaning workflow	2
3.3 Final classification and regression samples	3
3.4 Leakage control	3
3.5 Sensitive information	4
4 Evaluation Design	4
4.1 Train, validation, and test roles	4
4.2 Classification metrics	4
4.3 Regression metrics	4
5 Classification: Mortgage Approval Prediction	4
5.1 Problem definition	4
5.2 Feature engineering	5
5.3 Class-balancing strategies	5
5.4 Classification models	5
5.5 Classification results	5
5.6 Deep learning baselines for classification	6
5.7 Approaches that did not work well	7
5.8 Classification conclusion	7
6 Regression: Approved Loan Amount Prediction	7
6.1 Problem definition and scope	7
6.2 Raw and log-transformed targets	8
6.3 Linear models	8
6.4 Tree models	8
6.5 Boosting models	8
6.5.1 CatBoost	8
6.5.2 LightGBM	8
6.5.3 XGBoost	8
6.6 Deep tabular regression	8
6.7 Regression model journey	9
7 Final Regression Model and Test Results	9
7.1 Frozen boosting blend	9
7.2 Official test metrics	9
7.3 Post-test deep results	10
7.4 Why the deep-boosting ensemble was rejected	10
8 Error Analysis	11
8.1 Error by loan amount	11
8.2 Underprediction in the upper tail	11
8.3 Error concentration	11

8.4	Model disagreement	11
9	Explainability and Feature Interpretation	12
9.1	Why explainability is needed	12
9.2	Grouped permutation importance	12
9.3	SHAP and native tree importance	12
9.4	Local reference substitution	12
9.5	Most important feature groups	12
10	Fairness and Responsible Use	13
10.1	Fairness scope	13
10.2	Sensitive-feature comparisons	13
10.3	Why fairness conclusions are limited	13
10.4	Recommended responsible-use controls	13
11	Discussion	14
11.1	Why boosting models worked well	14
11.2	Why the two tasks had different winners	14
11.3	Value of simple baselines	14
11.4	Computational trade-offs	14
12	Conclusion	14

1 Introduction

1.1 Background

The Home Mortgage Disclosure Act requires many US financial institutions to report information about mortgage applications and lending activity. HMDA data include application outcomes, loan amounts, loan purposes, property characteristics, geographic information, applicant income, and selected demographic fields. The data are public after privacy protection steps. They are widely used for research on mortgage activity and lending patterns [1, 2].

Mortgage data are difficult to model for several reasons. First, the data contain both numbers and categories. Some categories, such as state or loan type, have a small number of values. Other columns, such as lender identity or detailed geography, may have hundreds or thousands of values. Second, some numeric variables have strongly skewed distributions. Income and loan amount can include a small number of very large values. Third, the approval target is imbalanced. Most rows in the final classification sample are approved applications. Finally, lending is a sensitive area. A model can have good average performance while still making larger errors for some groups or types of applications.

The project therefore had two goals. The first goal was predictive: build useful models for approval classification and loan amount regression. The second goal was analytical: understand why some models work better, where they fail, and what limits must be reported.

1.2 Project objectives

The project answers two main questions:

1. **Classification question:** Based on information available around the application stage, can a model predict whether the application will be approved?
2. **Regression question:** For an application that has already been approved, can a model estimate the approved loan amount?

These questions are related, but they are not the same. The classification target is a class label. The regression target is a continuous number. The useful metrics, class balance problems, model outputs, and error analysis are therefore different.

1.3 Why the problem is important

This problem is important for both technical and social reasons. Large lenders may process many applications, so automated analysis can help with research, monitoring, quality control, and workload planning. At the same time, mortgage models should not be treated like ordinary low-risk prediction tools. They can reflect patterns in historical data, including missing information, old business rules, geographic differences, and possible social inequality.

For this reason, the project does not present the models as automatic approval systems. The regression model also does not estimate default risk, interest rate, creditworthiness, or legal eligibility. The intended use is research and human-supported analysis. Final decisions should include domain experts, policy rules, and additional information that is not available in the HMDA file.

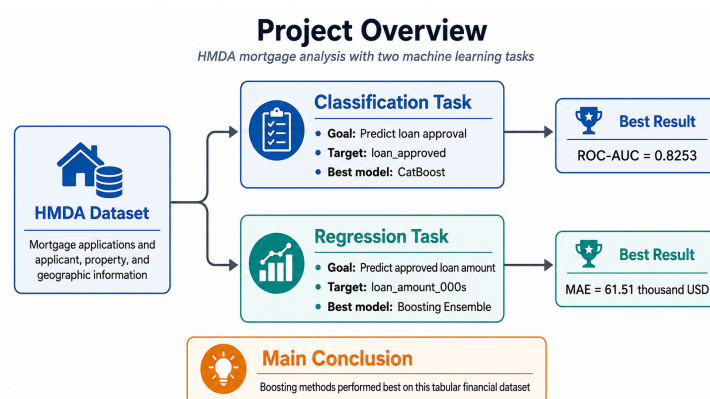


Figure 1. High-level view of the complete project. The first branch predicts approval, and the second branch predicts loan amount for approved applications.

2 Related Work

2.1 Classical models for tabular data

Logistic Regression is a common baseline for binary classification. It estimates the probability of the positive class from a weighted sum of input features. It is fast, easy to explain, and useful for checking whether the main signal is close to linear. For regression, Linear Regression and regularized versions such as Ridge, Lasso, and ElasticNet provide a similar baseline.

Decision trees learn simple rules by splitting the data. A single tree is easy to inspect, but it can be unstable. Random Forest reduces this problem by combining many trees trained with random samples and random feature choices [3]. The project used

both single-tree and forest models to measure how much non-linear structure was present.

2.2 Gradient boosting

Gradient boosting builds trees in sequence. Each new tree tries to reduce the errors left by the earlier trees. Modern boosting libraries add different methods for speed, regularization, sparse data, and categorical values.

XGBoost is designed as a scalable tree boosting system and includes sparsity-aware learning and efficient approximate split methods [4]. LightGBM uses histogram-based training and methods such as Gradient-based One-Side Sampling and Exclusive Feature Bundling to improve speed on large data [5]. CatBoost uses ordered boosting and special categorical processing to reduce target leakage and prediction shift when working with categories [6].

These methods are strong choices for mortgage data because the features are heterogeneous. The relation between income, loan purpose, property type, geography, and outcome is not expected to be a simple straight line.

2.3 Deep learning for tabular data

Deep learning is very successful for images, audio, and text, but the result is less clear for ordinary tabular data. Tabular neural networks must handle mixed data types, missing categories, small important groups, irregular decision boundaries, and features that have no natural spatial order.

The project tested standard multilayer networks and newer tabular models. FT-Transformer converts each feature into a token and applies Transformer blocks [7]. AutoInt uses self-attention to learn feature interactions [8]. TabNet uses sequential attention to choose useful features at different decision steps [9]. Wider benchmark studies show that tree models still often perform very well on medium-sized tabular problems and are usually faster to tune [10].

The purpose of the deep experiments in this project was not to assume that neural networks would win. Their purpose was to make a fair comparison and to learn whether embeddings and attention could add useful signal.

3 Data Source and Common Preparation

3.1 HMDA 2017 data

The source was the nationwide 2017 HMDA all-records file with labels. The raw file contained 14,285,496 rows and 78 columns. It included information about applications, loan actions, loan amount, loan type, purpose, property, location, lender, income, and selected demographic variables.

The full file was too large for simple in-memory processing on the available computer. The cleaning notebook therefore read and wrote the data in chunks of 200,000 rows. This design reduced memory use and made it possible to check the full file instead of using only a small early sample.

3.2 Cleaning workflow

The cleaning process had several stages. The main goal was to create a dataset that was smaller, safer, and easier to model without removing useful business information.

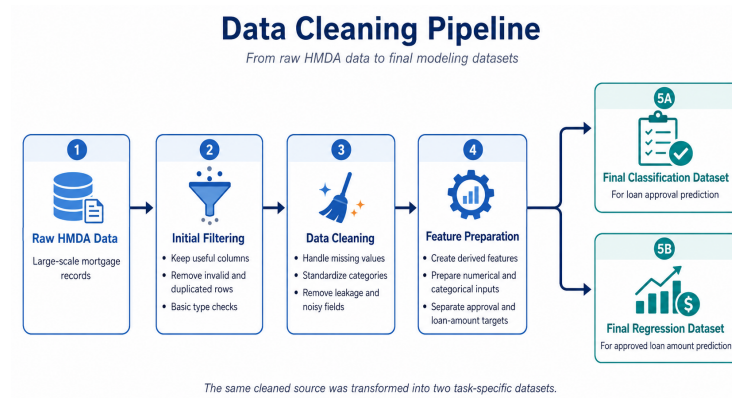
1. **Column removal.** Twenty-seven columns were removed from the original 78. These included administrative fields, detailed race fields with very high missingness, denial reasons, and rate spread. Denial reasons were removed because they are known after the outcome and would create target leakage.
2. **Required-value filtering.** Rows with missing or invalid values in important fields were removed. The required fields included applicant income, loan amount, location, tract income, population, minority population, and housing-unit information.
3. **Classification target creation.** Action codes 1, 2, and 8 were mapped to approved, while codes 3 and 7 were mapped to not approved. Other action codes were not used for the binary classification target.
4. **Duplicate and format checks.** Text values were stripped, row duplicates were checked, numeric values were converted carefully, and invalid category codes were reviewed.
5. **Range and extreme-value checks.** The notebook studied the smallest and largest loan and income values. Suspicious groups, such as values equal to 1 thousand dollars, were examined instead of being removed automatically.
6. **Final column selection.** Redundant code columns and model-unsafe fields were removed. Clear label columns, useful numbers, geographic context, and the target were kept.

Table 1 summarizes the main size changes. The final ready table contained 8,064,813 rows and 29 columns.

Table 1. Main stages in the common data preparation pipeline.

Stage	Rows	Columns	Main purpose
Raw nationwide HMDA file	14,285,496	78	Original 2017 all-records labeled data
After first column reduction	14,285,496	51	Remove administrative, sparse, and leakage-prone fields
After required-value filter	10,799,874	51	Keep rows with important financial and geographic values
After binary-action filter	8,074,536	52	Keep selected approval and rejection actions; add target
Final ready table	8,064,813	29	Strict checks and final feature selection
Classification modeling sample	500,000	29	Stratified sample with the original class ratio
Regression modeling sample	500,000	29	Approved applications only

Note: The duplicate-check notebook output and later strict-cleaning output came from separate saved steps. The report therefore uses the verified final ready-table count as the main cleaned-data size.

**Figure 2.** Common data-cleaning pipeline from the raw HMDA file to the two modeling datasets.

3.3 Final classification and regression samples

The classification sample contained 500,000 rows. It preserved the approval distribution of the cleaned full data. There were 401,226 approved rows and 98,774 not-approved rows, or about 80.25 percent and 19.75 percent.

The regression sample also contained 500,000 rows, but every row was approved. The regression modeling stages later retained 499,736 valid processed rows. A fixed split placed 399,788 rows in the training set and 99,948 rows in the test set.

Table 2. Final sample definitions for the two prediction tasks.

Task	Target	Rows	Positive/approved share	Scope
Classification	loan_approved	500,000	80.25%	Approved versus selected non-approved actions
Regression	loan_amount_000s	500,000 sample; 499,736 modeled	100%	Approved applications only

3.4 Leakage control

Data leakage happens when a model receives information that would not be available at the real prediction time, or when information from validation or test rows is used during training. The project used several rules to reduce this risk:

- Post-decision columns, such as denial reasons, were removed.
- Audit fields such as row_id and target bins were never model features.
- Imputation, scaling, rare-category grouping, and category mapping were kept inside complete pipelines.
- Model selection used training or validation evidence only.
- The regression test set stayed locked until the official final evaluation.
- Sensitive and non-sensitive comparisons used the same frozen model settings whenever possible.

3.5 Sensitive information

Sensitive information needs careful treatment. In the main classification pipeline, explicit race and ethnicity fields were removed from model inputs to reduce direct demographic dependence. The project still kept a separate audit path so that model behavior could be studied later.

In regression, matched experiments were created with and without a fixed group of sensitive features. The sensitive version added applicant and co-applicant ethnicity, race, and sex fields, together with two area-context fields. These comparisons were used only to study accuracy and group behavior. They were not used as proof of fairness, discrimination, or legal compliance.

4 Evaluation Design

4.1 Train, validation, and test roles

A fair experiment needs a clear role for each data part. The training set is used to fit the model. Validation data are used to choose features, target transformations, and hyperparameters. The test set is used only after choices are frozen.

For classification, the 500,000-row sample used a stratified 80/20 split. All reported final classification metrics were calculated on the held-out test part.

For regression, the saved split had 399,788 training rows and 99,948 test rows. Three fixed cross-validation folds were used inside the training data. The official test set was opened only at the locked final stage. Later deep-learning test results were marked as post-test descriptive results and were not allowed to replace the official model.

Important metric-scope rule

Out-of-fold, validation, official test, and post-test numbers do not have the same role. A smaller number from a later or easier scope cannot automatically replace an official result. Every comparison in this report states the evaluation scope.

4.2 Classification metrics

The classification task used several metrics:

- **ROC-AUC** measures how well the model ranks approved rows above non-approved rows across many thresholds. It was the main ranking metric.
- **Balanced accuracy** gives equal importance to both classes, even when one class is larger.
- **Precision** measures how often positive predictions are correct.
- **Recall** measures how many actual positive rows are found.
- **F1-score** balances precision and recall.

Accuracy alone was not used as the main metric because the data were imbalanced. A model could predict the majority class often and still have weak minority-class behavior.

4.3 Regression metrics

The main regression metric was Mean Absolute Error:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|. \quad (1)$$

Because the target is measured in thousands of US dollars, an MAE of 61.5 means an average absolute error of about 61,500 dollars.

Root Mean Squared Error gives more weight to large mistakes:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (2)$$

The project also used R^2 , RMSLE, mean signed error, P90 absolute error, top-decile MAE, and top-five-percent MAE. Mean signed error was defined as prediction minus target. A negative value means average underprediction. For this metric, distance from zero is more important than a simple lower-is-better rule.

5 Classification: Mortgage Approval Prediction

5.1 Problem definition

The classification target was:

$$\text{loan_approved} \in \{0, 1\}. \quad (3)$$

A value of 1 represented an approved application. A value of 0 represented a selected non-approved action. The model output was a probability, and a decision threshold converted that probability to a class.

This task was difficult because the class ratio was about four approved rows for every non-approved row. The dataset also included many categories and strongly overlapping feature distributions. Exploratory analysis showed that no single raw feature separated the two classes well.

5.2 Feature engineering

The classification pipeline added 17 derived features. The goal was not to create a very large feature space. The goal was to add financially meaningful relations that a model could use more easily. Important examples included:

- loan-to-income relations;
- applicant income compared with local median family income;
- owner-occupied housing share;
- log transformations for right-skewed numeric variables;
- a binary co-applicant indicator.

A random feature search also tried many automatic combinations. These random combinations gave only small and unstable gains. This supported the decision to keep the domain-based features instead of a large set of difficult-to-explain combinations.

5.3 Class-balancing strategies

Three training designs were tested:

1. **Under-sampled training.** The majority class was reduced until both classes had similar sizes.
2. **200k/200k training.** The majority class was reduced and the minority class was sampled with replacement to create a larger balanced set.
3. **Full training with class weights.** All available training rows were kept, and minority-class errors received more weight.

These designs tested an important trade-off. Under-sampling gives a balanced learning problem, but it throws away many majority examples. Full weighted training keeps more information, but the model must handle the imbalance directly.

5.4 Classification models

Table 3 summarizes the main model designs.

Table 3. Main classification model configurations.

Model	Preprocessing	Main settings and reason
Logistic Regression	Median imputation, standard scaling, one-hot encoding with rare-level grouping	saga solver, max_iter=1000; interpretable linear baseline
Random Forest	Encoded tabular features; no scaling required by trees	250 trees, minimum leaf size 20, depth chosen from 4 to 20; non-linear bagging baseline
CatBoost	Native categorical columns through cat_features	1,200 iterations, learning rate 0.05, depth 10, early stopping; strong categorical handling
LightGBM	Categorical columns stored as category type	700 trees, learning rate 0.04, 64 leaves, row and column sampling 0.85; fast boosting baseline

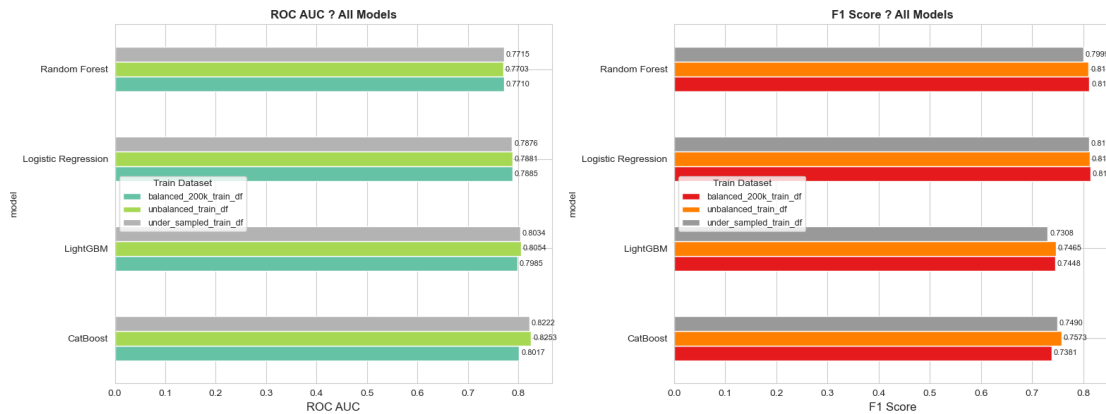
5.5 Classification results

Table 4 shows the retained classification results. The best ROC-AUC was 0.8253 from CatBoost on the full class-weighted training data. It also had the best balanced accuracy in the table.

Table 4. Classification results on the held-out test set.

Model	Training design	Bal. Acc.	Precision	Recall	F1	ROC-AUC
Logistic Regression	Under-sampled	0.7167	0.9086	0.7330	0.8114	0.7876
Logistic Regression	200k/200k	0.7185	0.9089	0.7373	0.8141	0.7885
Logistic Regression	Full weighted	0.7181	0.9091	0.7342	0.8123	0.7881
Random Forest	Under-sampled	0.7030	0.9034	0.7177	0.7999	0.7715
Random Forest	200k/200k	0.7028	0.8999	0.7402	0.8123	0.7710
Random Forest	Full weighted	0.7026	0.9001	0.7378	0.8109	0.7703
CatBoost	Under-sampled	0.7417	0.7284	0.7708	0.7490	0.8222
CatBoost	200k/200k	0.7291	0.7144	0.7635	0.7381	0.8017
CatBoost	Full weighted	0.7439	0.7196	0.7991	0.7573	0.8253
LightGBM	Under-sampled	0.7262	0.7189	0.7431	0.7308	0.8034
LightGBM	200k/200k	0.7235	0.6916	0.8069	0.7448	0.7985
LightGBM	Full weighted	0.7284	0.6999	0.7997	0.7465	0.8054
All-model majority vote	12 model instances	0.7356	0.7269	0.7547	0.7405	0.8158

The majority-vote ensemble had a ROC-AUC of 0.8158, which was lower than the best CatBoost result. This suggests that many models made similar mistakes. Voting can help when models are different, but it does not guarantee an improvement when their errors are strongly correlated.

**Figure 3.** Comparison of ROC-AUC and balanced accuracy for the retained classification models.

5.6 Deep learning baselines for classification

Six neural architectures were tested. Table 5 shows their results. FT-Transformer and AutoInt were the strongest deep classification models by ROC-AUC, but they still stayed below the best CatBoost result.

Table 5. Deep learning classification baselines.

Architecture	ROC-AUC	Average precision	F1
MLP with embeddings	0.7925	0.9302	0.8202
Wide & Deep	0.7896	0.9291	0.8362
FT-Transformer	0.7992	0.9323	0.8255
AutoInt	0.7989	0.9326	0.8244
NODE	0.7714	0.9219	0.8041
TabNet	0.7527	0.9130	0.8390

The deep models required much more training time per epoch in the project. Their ROC-AUC values were also lower than CatBoost and LightGBM. For this dataset, the extra complexity did not create enough predictive value.

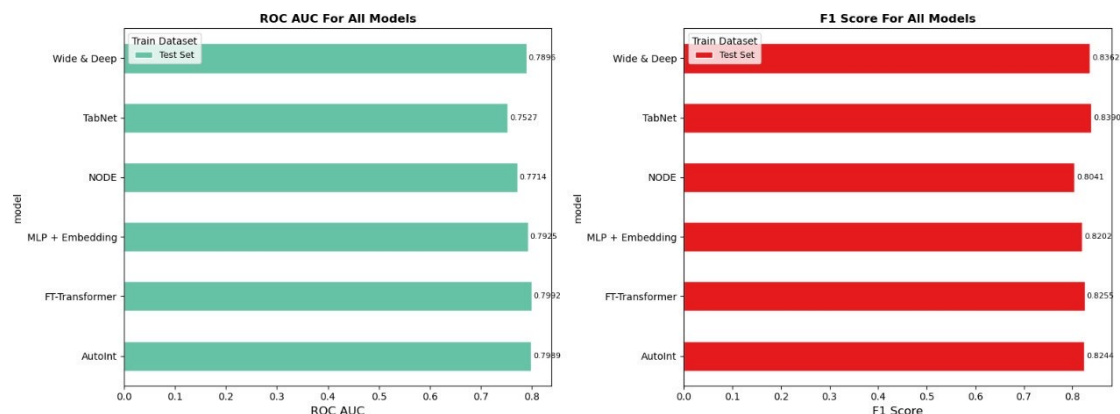


Figure 4. ROC-AUC comparison for the six deep classification architectures.

5.7 Approaches that did not work well

A custom AdaBoost-style method was also developed. At each step, it tried to choose features with high mutual information and low similarity to earlier features. It also made random feature additions and removals. The method reached only about 65 percent accuracy, and its validation AUC improvement became very small after several rounds. The random search was slow and not fully stable across seeds, so this approach was not continued.

Feature ablation gave another useful result. Removing one feature at a time changed AUC by only about ± 0.001 in the largest cases. This means the classification signal was spread across many features. No single column explained the whole task.

5.8 Classification conclusion

The final classification choice was the full weighted CatBoost model. It kept all training information, handled categorical features directly, and reached the best ROC-AUC. Logistic Regression remained a useful interpretable baseline. LightGBM was also competitive, while Random Forest and the tested deep models were weaker on the main ranking metric.

6 Regression: Approved Loan Amount Prediction

6.1 Problem definition and scope

The regression target was `loan_amount_000s`. The unit is thousands of US dollars. For example, a target value of 250 means about 250,000 dollars.

Every regression row was an approved application. The model did not predict approval, rejection, default, creditworthiness, or interest rate. Its prediction is meaningful only inside the approved-application population represented by the data.

The input features described several parts of the application:

- applicant financial information, such as income;
- loan structure, such as type, purpose, and lien status;
- property and occupancy information;
- geographic context, such as state, county, metropolitan area, and region;
- local economic context, such as median family income and tract-income ratios.

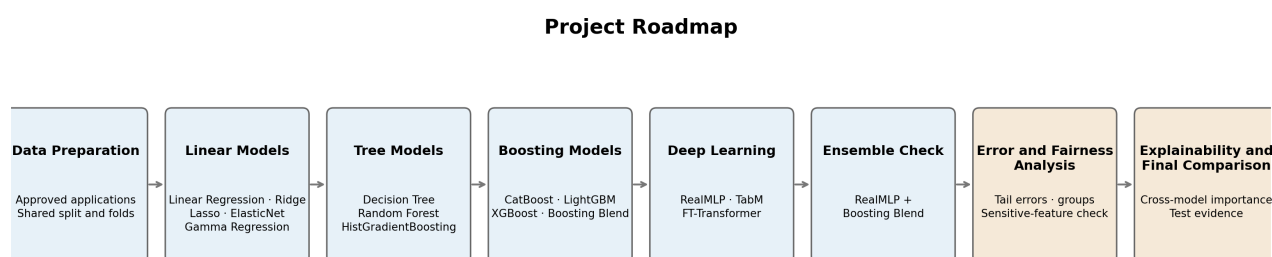


Figure 5. Regression project roadmap from linear baselines to boosting, deep learning, error analysis, and explainability.

6.2 Raw and log-transformed targets

The loan amount had a right-skewed distribution. A small number of very large loans created a long upper tail. The project compared two target modes.

In raw mode, the model learned the original value y . In log mode, the target was transformed as:

$$z = \log(1 + y), \quad \hat{y} = \exp(\hat{z}) - 1. \quad (4)$$

All reported metrics were calculated after returning predictions to the original thousand-dollar scale. The log transformation helped some models, but not all. CatBoost, XGBoost, and FT-Transformer preferred the log target. LightGBM and RealMLP performed better with the raw target.

This result is important because target transformation is a model choice, not a universal rule. Log training can improve stability, but it can also make a model more conservative for very large loans.

6.3 Linear models

The linear stage included Linear Regression, Ridge, Lasso, ElasticNet, and Gamma Regression. Numeric features were imputed and scaled. Categorical features were one-hot encoded. Every learned preprocessing step stayed inside the model pipeline.

Lasso was the strongest retained linear model, with an out-of-fold MAE of about 72.230 thousand dollars without sensitive features. Ridge and ElasticNet were close. Gamma Regression performed very poorly, with an out-of-fold MAE above 9,000. This showed that its distribution and link assumptions did not fit the target well.

The linear stage was still valuable. It proved that the complete pipeline worked and created a clear baseline. It also showed that the problem had strong non-linear structure.

6.4 Tree models

A single Decision Tree improved only a little over the linear models. Random Forest was better, but the first large configuration was too slow. A smaller 64-tree configuration was selected after runtime-aware screening. Its out-of-fold MAE was about 69.142 without sensitive features.

HistGradientBoosting was much stronger. It used binned numeric values and sequential tree fitting. Its out-of-fold MAE was about 64.276 without sensitive features. This was a large improvement over the linear and bagging baselines.

6.5 Boosting models

Three modern boosting families were developed separately.

6.5.1 CatBoost

CatBoost used native categorical features and grouped rare categories inside the pipeline. The final non-sensitive configuration used depth 6, learning rate 0.05, L2 regularization 20, random strength 1, 2,000 iterations, and the log target. Its final-selection validation MAE was 63.376185.

6.5.2 LightGBM

LightGBM used training-fitted frequency encoding for high-cardinality categories and ordinal encoding for other categories. Its final design used the raw target, 47 leaves, learning rate 0.05, 1,500 trees, minimum child samples 50, and L2 regularization 5. Its final-selection validation MAE was 64.456751.

6.5.3 XGBoost

XGBoost used sparse one-hot encoding for controlled categories and frequency encoding for high-cardinality fields. One extra income-ratio feature passed the independent confirmation step. The final model used the log target, depth 6, learning rate 0.05, minimum child weight 10, L2 regularization 5, and 982 trees. Its final-selection validation MAE was 64.221924.

6.6 Deep tabular regression

Three deep families were studied in the main regression path.

RealMLP learned numeric processing and category vocabularies from training rows only. The winning mode used the raw target, dropout 0.15, one repeat, and a fixed 30-epoch policy. Its final-selection validation MAE was 63.923004.

TabM used piecewise-linear embeddings for numeric features and learned embeddings for categories. Its best screening MAE was 66.123240, so it did not continue to the final deep stage.

FT-Transformer converted features into tokens and used Transformer blocks. The log target worked better. Its final-selection validation MAE was 64.458782.

The deep results were competitive, but CatBoost and the final boosting blend remained stronger for the main MAE objective.

6.7 Regression model journey

Table 6 gives a compact view of the model journey. The evaluation scope is included because these values must not be treated as one direct leaderboard.

Table 6. Regression model journey. MAE values are comparable only within the same evaluation scope.

Model	Target mode	MAE	Scope	Decision
Linear Regression	log1p	72.410	OOF training	Baseline
Ridge	log1p	72.311	OOF training	Stopped
Lasso	log1p	72.230	OOF training	Linear representative
Decision Tree	log1p	71.800	OOF training	Tree baseline
Random Forest	log1p	69.142	OOF training	Valid but slower/weaker
HistGradientBoosting	log1p	64.276	OOF training	Tree representative
CatBoost	log1p	63.376	Final validation	Boosting component
LightGBM	raw	64.457	Final validation	Boosting component
XGBoost	log1p	64.222	Final validation	Boosting component
RealMLP	raw	63.923	Final validation	Best deep model
FT-Transformer	log1p	64.459	Final validation	Second deep model
Boosting blend	Mixed	61.512	Official test	Official final model

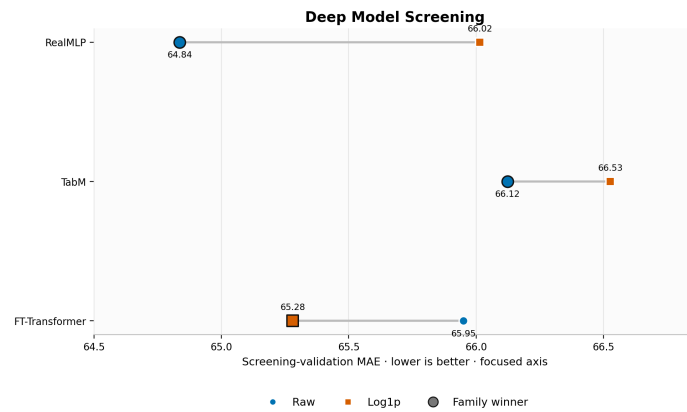


Figure 6. Regression performance journey with evaluation scopes shown separately.

7 Final Regression Model and Test Results

7.1 Frozen boosting blend

The official final model combined three boosting predictions after all outputs were returned to the original target scale:

$$\hat{y}_{\text{blend}} = 0.60\hat{y}_{\text{CatBoost}} + 0.20\hat{y}_{\text{LightGBM}} + 0.20\hat{y}_{\text{XGBoost}}. \quad (5)$$

The weights were selected with non-sensitive validation predictions before the test set was opened. The same weights were used for matched sensitive analysis. No test result was used to change the weights.

The blend idea was simple. CatBoost, LightGBM, and XGBoost used different category processing and tree-building methods. Their mistakes were similar, but not identical. A weighted average could reduce some model-specific errors.

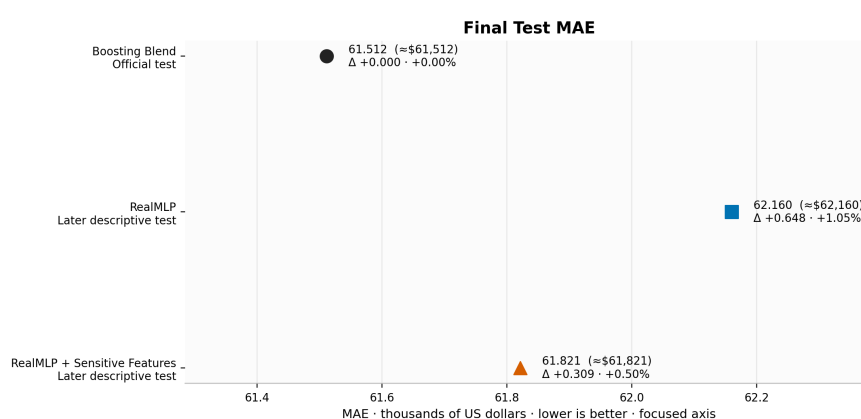
7.2 Official test metrics

The official non-sensitive blend reached the results in Table 7 on 99,948 locked test rows.

Table 7. Official regression test metrics for the frozen boosting blend.

Metric	Value	Plain meaning
MAE	61.511631	About 61,512 dollars average absolute error
RMSE	125.788460	Larger mistakes receive more weight
R^2	0.712206	Explained variation; not a percent accuracy
RMSLE	0.383861	Error on a logarithmic scale
Mean signed error	-9.554174	Average underprediction
Top-decile MAE	195.498099	Error for the largest target decile
Top-five-percent MAE	287.063969	Error for the largest 5 percent of target values

The MAE was the main selection metric, but the other values show why one number is not enough. RMSE was much larger than MAE because a small group of difficult cases had very large errors. The negative signed error showed a general underprediction direction.

**Figure 7.** Final test MAE comparison. Official and post-test roles should be shown clearly.

7.3 Post-test deep results

RealMLP was evaluated later on the same test rows. The non-sensitive RealMLP had MAE 62.159745, while the sensitive version had MAE 61.821114. These values are useful for description, but they are post-test extensions. They cannot replace the official blend without a new independent holdout.

Table 8. Official and post-test regression results on the same 99,948 rows.

Candidate	Role	MAE	RMSE	R^2	RMSLE	Signed error	Top 5% MAE
Boosting blend	Official pre-registered test	61.511631	125.788460	0.712206	0.383861	-9.554174	287.063969
RealMLP	Later descriptive test	62.159745	125.651422	0.712833	0.389685	-7.013757	278.545533
RealMLP + sensitive	Later accuracy-only test	61.821114	125.020164	0.715711	0.389633	-6.440910	277.236066

The post-test RealMLP models had slightly better RMSE and tail metrics, but the official blend had the best official MAE. This difference shows that model quality depends on the metric and the evaluation role.

7.4 Why the deep-boosting ensemble was rejected

A 50/50 combination of RealMLP and the frozen boosting blend was tested on aligned validation predictions. It improved MAE by 0.440155 percent and improved top-decile MAE by 1.221917 percent. A paired bootstrap interval also supported the MAE improvement.

However, the acceptance rule was defined before seeing the result. RMSE was not allowed to become more than 0.25 percent worse. The ensemble made RMSE 0.345731 percent worse, so it failed the rule and was rejected. It did not receive a test prediction.

This was an important project decision. A small improvement in the main metric was not enough when large-error behavior became worse than the allowed limit.

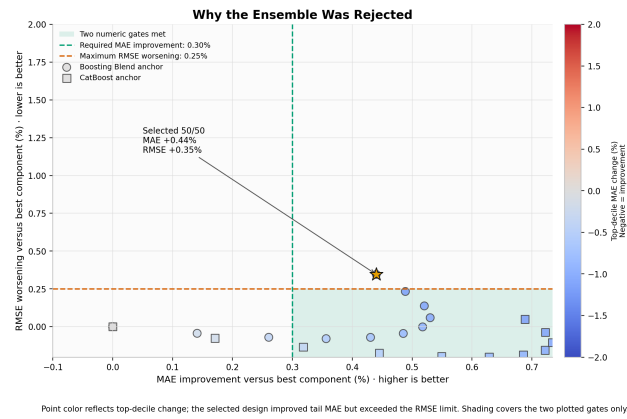


Figure 8. Validation trade-off that explains why the 50/50 deep-boosting ensemble was rejected.

8 Error Analysis

8.1 Error by loan amount

Average performance can hide difficult regions. The regression test rows were therefore divided by the true loan amount. Error increased as the loan amount increased. The highest target decile was much harder than the other deciles.

This pattern is reasonable from a modeling view. Large loans are less common, so the model sees fewer similar examples. They may also depend on information that is not present in the HMDA dataset, such as detailed credit history, assets, debt-to-income ratio, and loan-to-value ratio.

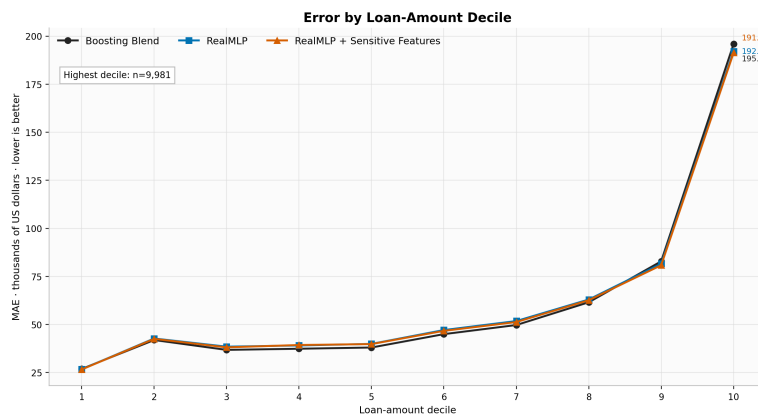


Figure 9. Mean absolute error by true loan-amount decile. The highest decile is the most difficult region.

8.2 Underprediction in the upper tail

The official blend had a mean signed error of -9.554 thousand dollars. Underprediction became stronger in the target tail. About 79 percent of rows in the top decile were underpredicted, and about 83 percent of rows in the top 5 percent were underpredicted.

This matters in practice. A model that is usually close but systematically low for the largest loans can create planning or review problems. High-value cases should therefore receive additional human review and uncertainty checks.

8.3 Error concentration

The worst 10 percent of official-blend cases produced about 42.71 percent of the total absolute error. This means that a relatively small group of difficult rows explained a large part of the overall loss.

A useful operational design could include two paths. Ordinary cases could receive the standard model estimate. Cases with very large predicted amounts, unusual categories, or high uncertainty could be sent to a specialist for review.

8.4 Model disagreement

The official blend and non-sensitive RealMLP predictions had a Pearson correlation of about 0.984. Their general movement was similar. However, the mean absolute difference between their predictions was about 16.69 thousand dollars. Some rows had much larger disagreement.

High correlation therefore does not mean that two models are identical. Disagreement can help find unusual cases, possible data-quality problems, or regions where model uncertainty is high.

9 Explainability and Feature Interpretation

9.1 Why explainability is needed

A good score is not enough for a financial model. We also need to know what information the model uses. Explainability does not prove cause and effect, but it can show model dependence and help find suspicious behavior.

The project used three complementary methods:

1. grouped permutation importance;
2. saved native tree importance and SHAP evidence;
3. local reference substitution for selected cases.

9.2 Grouped permutation importance

Grouped permutation importance changes one feature or one meaningful feature group by shuffling it across rows. If the prediction error increases, the model depends on that information.

The final comparison used the same 2,000 saved test rows for the official blend, RealMLP, and sensitive RealMLP. Using the same rows made the ranking comparison clearer. A limitation is that correlated features can share or hide each other's importance.

9.3 SHAP and native tree importance

CatBoost, LightGBM, and XGBoost had saved native importance and SHAP evidence. SHAP describes how a feature moves one prediction away from a reference value. The three component models did not always use the same output scale, so their raw SHAP values were not added together as if they were directly equal. The official blend comparison used grouped permutation on the original target scale.

9.4 Local reference substitution

For 20 selected public cases, one feature or feature group was replaced by 40 reference values. The change in prediction was recorded. The final evidence contained 64,800 local effects with the mean, standard deviation, minimum, maximum, and mean absolute effect.

This method is useful for sensitivity analysis, but it is not SHAP, it is not additive, and it is not a real policy intervention. Replacing one feature can create combinations that are rare in real data.

9.5 Most important feature groups

Applicant income and lien status were leading feature groups across the final regression models. Occupancy, local income context, loan purpose, geography, and lender identity also appeared.

Table 9. Shared high-importance regression feature groups. Lower rank means stronger observed dependence.

Feature or group	Blend rank	RealMLP rank	Sensitive RealMLP rank	Limited interpretation
Applicant income	1	1	1	Main shared financial signal
Lien status	2	2	2	Loan-priority structure
Owner occupancy	3	4	3	Property-use context
Median family income	4	7	7	Local income context
Tract income ratio	5	5	5	Applicant-area relation
Loan purpose	6	8	8	Purpose of borrowing
State	7	19	14	Broad geography
Metropolitan area	8	3	4	Local market context
County	9	11	13	Detailed geography
US region	10	6	6	Large geographic region
Respondent or lender identity	13	9	9	Institution pattern; possible proxy
Sensitive context block	—	—	11	Aggregate dependence only
Explicit sensitive identity block	—	—	19	Does not prove discrimination

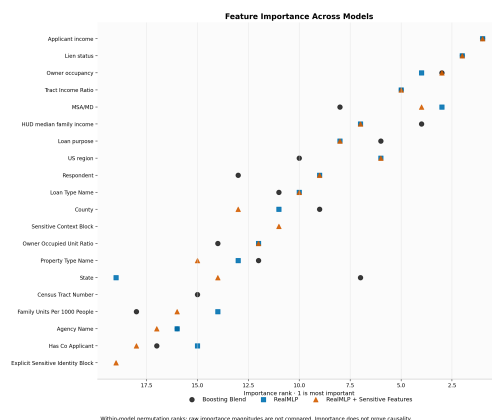


Figure 10. Cross-model grouped feature-importance comparison on the same saved sample.

The main conclusion is not that income or geography causes the loan amount. The conclusion is that the fitted models used these variables strongly in this dataset.

10 Fairness and Responsible Use

10.1 Fairness scope

Fairness was considered in both tasks, but the available evidence had important limits.

The classification task included both approved and selected non-approved applications. Explicit race and ethnicity variables were removed from the main predictive input. This reduces direct use, but it does not remove all proxy information. Geography, lender, income, and property variables can still be related to demographic patterns.

The regression task included approved applications only. It therefore cannot answer whether people from different groups had equal access to approval. It can only study prediction error among applications that were already approved.

10.2 Sensitive-feature comparisons

Matched sensitive and non-sensitive regression models used the same frozen configurations. In some models, sensitive features slightly improved MAE. This is an accuracy observation only. It does not prove that using sensitive features is fair, and it does not prove that removing them is fair.

The final descriptive fairness analysis reported group error gaps and used suppression rules for small or unstable groups. The largest primary-group MAE gap was 39.687590 thousand dollars. This value should be treated as a monitoring signal, not a legal threshold or causal result.

10.3 Why fairness conclusions are limited

Several reasons prevent a strong fairness claim:

- HMDA does not include every factor used in real underwriting.
- Historical targets may contain past business and social patterns.
- Regression covers approved applications only.
- Small groups can have unstable metrics.
- Group differences can be caused by different target distributions.
- Feature importance and subgroup error do not prove causality.
- Legal compliance requires more information and domain review.

10.4 Recommended responsible-use controls

A real deployment would need at least the following controls:

- human review for high-value or high-uncertainty cases;
- separate monitoring of MAE, RMSE, tail MAE, and signed error;
- group-level performance monitoring with minimum sample rules;
- checks for missing-category and feature-distribution drift;
- clear documentation of target scope and target units;

- a process for investigating large disagreements and complaints;
- no use for automatic approval, rejection, pricing, or legal certification.

11 Discussion

11.1 Why boosting models worked well

Boosting models were strong in both tasks. There are several likely reasons.

First, the data were tabular and mixed numeric and categorical values. Tree splits can model irregular relations without a fixed linear formula. Second, many features were weak alone but useful together. Boosting can build these interactions gradually. Third, CatBoost handled categorical values directly, while LightGBM and XGBoost used model-specific encodings. Fourth, the number of rows was large enough to support complex trees but still small enough for careful validation.

The results also match wider tabular-learning research, which often finds that well-tuned tree ensembles are difficult to beat on ordinary structured data [10].

11.2 Why the two tasks had different winners

The classification winner was one CatBoost model. The regression winner was a blend of three boosting models. This difference is reasonable.

For classification, CatBoost produced the best ranking and the majority vote could not add enough diversity. For regression, the three boosting models used different encodings, target modes, and tree-building methods. Their numeric prediction errors were different enough that a weighted average improved MAE.

11.3 Value of simple baselines

The weaker models were still useful. Logistic Regression and Lasso showed the linear performance level. Decision Tree showed the effect of simple non-linearity. Random Forest showed the value and cost of bagging. These baselines made it possible to explain why the later models were needed.

A project that reports only the final model gives less information. The full model journey shows that the improvement came from non-linear structure, better category handling, target transformation, and model diversity.

11.4 Computational trade-offs

The project used bounded samples, sequential heavy fits, checkpoints, and cache validation because the full workflow was expensive. Deep models were especially costly. Some early Random Forest and deep-learning configurations also had runtime or protocol problems.

These experiences are part of the result. A model is not useful only because it has a good metric. It should also be trainable, reproducible, reloadable, and understandable within the available resources.

12 Conclusion

This project built a complete machine learning study for mortgage data. It started with more than 14 million raw HMDA rows and created two clean modeling samples. The first task predicted application approval. The second task predicted the loan amount for approved applications.

For classification, CatBoost trained with full weighted data gave the strongest result. It reached ROC-AUC 0.8253 and balanced accuracy 0.7439. The majority-vote ensemble and six deep architectures did not improve the main ranking metric.

For regression, model performance improved as the project moved from linear models to tree models and then to boosting. The official final model combined CatBoost, LightGBM, and XGBoost. It reached an official test MAE of 61.511631 thousand dollars and R^2 of 0.712206. Deep tabular models were competitive, but the best deep-boosting combination failed a pre-defined RMSE rule and was rejected.

The error analysis showed that large loans were the most difficult cases. Underprediction was stronger in the upper tail, and a small group of rows created a large share of total error. Explainability showed strong model dependence on applicant income, lien status, occupancy, local income context, loan purpose, and geography.

The main lesson is that a strong financial machine learning project needs more than one good score. It needs careful data preparation, leakage control, fixed evaluation roles, honest reporting of failed ideas, error analysis, fairness limits, explainability, and clear human-use boundaries. Gradient boosting was the best practical choice for this dataset, but the final models should support human analysis rather than replace high-stakes decision makers.

References

- [1] Consumer Financial Protection Bureau. *Home Mortgage Disclosure Act (HMDA) Data*. <https://www.consumerfinance.gov/data-research/hmda/>

- [2] Federal Financial Institutions Examination Council and Consumer Financial Protection Bureau. *Availability of 2017 Data on Mortgage Lending*, 2018. <https://www.consumerfinance.gov/archive/newsroom/ffiec-announces-availability-2017-data-mortgage-lending/>
- [3] L. Breiman. Random Forests. *Machine Learning*, 45(1):5–32, 2001. <https://doi.org/10.1023/A:1010933404324>
- [4] T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016. <https://arxiv.org/abs/1603.02754>
- [5] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems 30*, 2017. <https://papers.nips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>
- [6] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin. CatBoost: Unbiased Boosting with Categorical Features. In *Advances in Neural Information Processing Systems 31*, 2018. <https://arxiv.org/abs/1706.09516>
- [7] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko. Revisiting Deep Learning Models for Tabular Data. In *Advances in Neural Information Processing Systems 34*, 2021. <https://arxiv.org/abs/2106.11959>
- [8] W. Song, C. Shi, Z. Xiao, Z. Duan, Y. Xu, M. Zhang, and J. Tang. AutoInt: Automatic Feature Interaction Learning via Self-Attentive Neural Networks. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, 2019. <https://arxiv.org/abs/1810.11921>
- [9] S. O. Arik and T. Pfister. TabNet: Attentive Interpretable Tabular Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(8), 2021. <https://arxiv.org/abs/1908.07442>
- [10] L. Grinsztajn, E. Oyallon, and G. Varoquaux. Why Do Tree-Based Models Still Outperform Deep Learning on Tabular Data? In *Advances in Neural Information Processing Systems 35*, 2022. <https://arxiv.org/abs/2207.08815>